Manage your time wisely! Projects are weighted relatively lightly in this course, ZOBOS is worth $< 4\%$ of your course grade. Depending on where you are now and your expectations of a course grade, you could legitimately just not do ZOBOS.

Downsides to NOT doing ZOBOS

- CZAR *might* be slightly harder, depending on your definition of "hard". CZAR can be effectively "bootstrapped" from ZOBOS. Without ZOBOS you'll have to write a little more code --- but not nearly as much code as ZOBOS.
- You don't get "Two compilers written" bragging rights at the end of the semester.

Upsides to NOT doing ZOBOS

- CZAR typically has some extra-credit associated with it, so you can make up part of the $< 4\%$ point loss incurred by avoiding ZOBOS.
- CZAR is focused on code generation, if that's what you're really interested in understanding (granted at the most basic levels), then maybe this is the right choice.
- You do get "One compiler, a scanner and a code generator" bragging rights (in order WRECK, (WRECK + NFAMATCH + LUTHOR), CZAR).
- CZAR will be due *during finals week*. So if this decision has more to do with other course projects due at the end of the semester; skipping ZOBOS can be an effective time management tool. (And I'm happy to accept ZOBOS submissions during finals week, just communicate appropriately and let me know whats what.)

- The zlang page has language details, nuances, and caveats you might be interested in.
- You can download all the resources used in this writeup: zobos-resources.zip or zobos-resources.tar.bz2.
  The tarball contains source files (`.src`, `.syn`), token streams (`.tok`), AST PDFs, and verbose and `OUTPUT` filtered `stdout` listings (`.vrb` and `.out`) respectively. **Read the `README.txt`!**
- Here is the change log for this assignment write-up. I will try to be descriptive in my log messages.
- You can also subscribe to this page and receive Emails when changes are made.

In this assignment will you will implement a syntax checking SLR(1) parser, a symbol table(s), and perform semantic analysis on a simple programming language grammar. The name of your application must be `ZOBOS` (case sensitive).

Input

`ZOBOS` will accept three command line parameters:

Argument	Value	Description
1	`program.tok`	The filename containing a **token stream** to be parsed, the format is identical to that produced by LUTHOR
2	`ast.dat`	The filename for storing your AST
3	`symtable.dat`	The filename for storing symbol tables when instructed.

Coding

`ZOBOS` will do do the following:

1. Your compiler will parse a token stream (the output of LUTHOR) with the LR knitting needles algorithm (pseudo code here), using this SLR table (the format is described here) and the zlang.cfg grammar (a pure CFG file is here, without commentary). Syntax error messages will be produced in a consistent manner to facilitate grading.

    - If program source has a syntax error, the remaining steps for `ZOBOS` are not to be performed and `ZOBOS` should have an **exit status of 2**.

2. Either during the parse using **syntax directed translation** procedures (on production rule reduction, show_lr-knitting-with-sdt.pdf, lga-sdt.pdf) or after the parse using **visitor patterns**, or a combination of these methods; `ZOBOS` will construct an *abstract syntax tree* with the following properties
    a. all `EXPR` (sub)trees will be simplified so that
        i. leaves are either literals (`intval`s, `floatval`s) or variables (identifiers holding a value, not function names)
        ii. the root and internal nodes are either one of the operations associated with the non-terminals `BOOLS`, `PLUS`, `MULT`, `UNARY`, or `CAST`, a `FUNCALL` node or a domain node.
    b. control structures `IF`, `IFELSE`, `WHILE`, `DOWHILE`, `STEPS`, `SOLOSTMT`, and `BRACESTMTS` will be simplified **in the spirit** of textbook figure 7.15 (lga-sdt.pdf) --- essentially lose the terminals, keep the functional children (blocks of statements, predicate expressions).

    Simplification of all other grammar constructs are at the student's discretion and inclination. They will not be graded.
    There are example `.src` and AST PDFs in the `zobos-`

> `resources` archives for download.
>
> Outside of the rules listed above, **you are not required** to duplicate the ASTs provided, they serve as examples!

3. `ZOBOS` will write to disk a simple tree representation of its AST.
4. `ZOBOS` will then use a scope respecting symbol table and visitor pattern(s) to perform several semantic checks on the variable, expression and function use within the parsed program source.
    a. Warnings or error messages will be emitted in a consistent and simple manner to facilitate grading.
    b. When the special `EMIT` form of `symtable` is encountered during this process, your `ZOBOS` will write the symbol tables for all scopes to disk in a simple format to facilitate grading.
    c. When a special `domain` node is encountered, the domain of its expression tree child should be emitted in a in a format similar to ERRORs *to facilitate grading*. The possible domain values are `int`, `float`, `string` or `bool`.
5. Lastly, `ZOBOS` will have an **exit status of 0** if there were **no semantic errors** detected in the program, and `exit(3)` otherwise.

> `WARNING` and `DOMAIN` messages are not `ERROR` messages

The ZOBOS language

The language compiled for this project is "C-like", but with some caveats and nuances. See this page for details.

> You may either hard code the LR parsing table and grammar rules into your `ZOBOS` application or read `zlang.lr` and the grammar rules from disk on each invocation. **If the latter**, include `zlang.lr` and (or) `zlang-pure.cfg` in your submitted archive along side `Build.sh` or the `Makefile`; which is where your `ZOBOS` application (script) should be after your submission is built. This means the files will be in the current working directory (`./`) when your `ZOBOS` is invoked by the `grader.sh` script. See this page to recall submission details.

Syntax, DOMAIN and Semantic Checks

Your `ZOBOS` compiler will inspect the `program.tok` for these types of errors and inconsistencies.

> Keep in mind that the CS culture often clumps all errors into the *syntax error* pigeonhole. Technically, **syntax errors** occur only during the parse when an invalid pattern of terminals is detected. The other errors that compilers find for us, and *most of them* in this project, are

semantic errors and inconsistencies.

Our language does not have "first class" functions, and functions won't be treated as identifiers holding a "value". The language has data types `bool`, `int`, `float` and `string`; and then it has functions. Think C/C++, not Python, JavaScript, Scheme or Haskell.

Type	ID	Description	Terminal Location to report (line and column)
All submissions			
SYNTAX	SYNTAX	Any syntax error during the parse	Token causing error, or last token successfully processed
DOMAIN	DOMAIN	`domain` language instruction, see here for more details.	The location of the `domain` terminal in the source.
WARN	UNUSED	The identifier is not used (**correctly or incorrectly**) in an expression or assignment **anywhere** within its scopes. The `returns` variable of a function is **implicitly** used (since the function will return the value it stores). In other words, the identifier is not found *to the right* of `=`, or in an `emit` statement (see emit details).	Location of identifier in its declaration
ERROR	CONV	Value conversion error (see the conversion errors table and EMIT and RAND details)	Location of the terminal at the root of the `EXPR` tree, which is likely the `assign` terminal (`=`) or a literal value. **Not a conversion issue related to function calls,** those are FNCONV errors.
Graduate course submissions			
WARN	REIDENT	Attempting to re-declare an identifier in the same scope.	**Each** occurrence of identifier in re-declaration. This includes duplicating parameter names in function headers (see here for the scope values associated with functions).

WARN	UNINIT	Using a **declared** identifier in an expression before it has been initialized with a value. In other words the variable is not found *to the left* of `=` **or** to the right of a `rand` statement before it is used (see rand for details). **Undeclared identifiers don't have UNINIT warnings.**	Location of identifier in expression
ERROR	CALL	Invoking a variable identifier as a function, using the wrong number of arguments in a function call	Location of the function identifier being invoked
ERROR	EXPR	Expression tree operand error (see the expression errors table)	Location of the `PLUS`, `TIMES`, `mod` or `UNARY` terminal operating on the erroneous value or variable
Unused this semester			
WARN	CONST	**Not required this semester**	
WARN	NOIDENT	**Not required this semester**	
ERROR	FNCONV	**Not required this semester**	

> Many types of semantic checks could be performed, not all are used for each ZOBOS project per semester. You might read elsewhere in the write-up conditional statements such as *"If required by the semester write-up, such-and-such would be a FOOBAR warning."* Depending on the semester these statements are either important or can be safely ignored.
>
> The table above is the **authoritative semester list of warnings and errors your ZOBOS should report**; use it to guide your implementation.

Symbol Table Contents and Functions as (sort of a) Type

While functions are not first class objects in our language, we will adopt semantics around function prototypes and definitions so that `ERROR` and `WARN` messages, as well as the symbol table contents, don't have to be specialized for function types. A symbol table should have the following (minimum) attributes per entry for `ZOBOS` success:

- location (line and column reported for the identifier token, see here for function definitions and prototypes),
- identifier (name)
- type (see here for function "types"),
- a `const` flag,

- a "used" or "unused" flag,
- an "initialized" or "uninitialized" flag.

> The last three symbol attributes, `const`-ness, used vs unused, initialized vs uninitialized, may not be needed in your implementation. It depends on the types of semantic checks required for this semester's ZOBOS project.

When an identifier for a function is encountered, its value in the symbol table depends on the source context:

1. When a function prototype is encountered, the symbol is considered **not `const`**, but **already initialized**.
2. When a function definition is encountered the symbol has its `const` flag set (**or** is created with its `const` and `initialized` flag true).

These semantics mean your `ZOBOS` does not have to have different logic for processing variables versus functions (aside from the initial management of function symbols just described). The lion's share of function specific code producing output is for `CALL` error messages.

Conversion Errors (CONV and FNCONV)

Attempting to perform these types of data storage causes a `CONV ERROR`.

Value Type	Stored into ...
`string`	`int`, `float` or `bool`
`float`	`int`, `bool` or `string`
`bool`	`float` or `string`
`int`	`bool` or `string`
`string`, `float`, `bool` or `int`	A function identifier (see here for the flip-flop)

`FNCONV` errors are detected by the same rules, but only when the conversion issue is for function parameter values or values returned by functions.

EMIT and RAND Conversion Errors

The `emit` and `rand` statements in zlang have particular type semantics that must be met, otherwise a `CONV` error should be reported.

Statement	Type Semantics
`emit id, offset, length`	The `id` argument of `emit` must be a `string` type, `offset` and `length` must be `int` types (a variable, integer expression).
`emit (expr)`	`expr` must be of type `bool`, `int` or `float`

`rand id`	`id` must be a variable of type `bool`, `int` or `float`

`FNCONV` errors associated with `emit` parameters occur when a non-integer function return value is uses for the offset or length argument.

All other argument type patterns for `emit` and `rand` should generate `CONV` errors, the location value should be the line and column of the offending identifier or the line and column of the root operation in the case of an arithmetic expression.

Some examples (`str` is a `string`, `flt()` is a function returning a `float`, `bln()` is a function returning a `bool`):

Statement	ERROR	Reason
`emit str, 3, int(bln());`	None	No error, `3` is a literal integer and the Boolean return value is cast to an integer
`emit str, 3, (bln());`	`:ERROR: 7 15 :FNCONV:`	The immediate use of a `bool` return value from a function for the third parameter (needs to be an integer)
`emit str, (flt()), (2+3);`	`:ERROR: 3 12 :FNCONV:`	The immediate use of a `float` return value from a function for the second parameter
`emit str, ((flt()) + 3.14), 8;`	`:ERROR: 6 20 :CONV:`	Use of a `float` valued expression for the second parameter

The last case is not an `FNCONV` error because the floating point function result can be added to `3.14` without issue. It could also be thought of as the function result is not being erroneously stored in a memory space associated with a non-float type --- so there is no data conversion issue. Its the floating point *expression* result that creates the data conversion issue, hence a `CONV` error.

)>> ||

```
OUTPUT :ERROR: 2 9 :FNCONV:
REASON cannot store expression value of type float into parameter of type
int
```

||So it is the function identifier with the parameter being mis-valued, not the function identifier providing the wrong value. See the CONV and emit sections for remaining `FNCONV` implementation details. }}}

Expression Value Propagation Rules and Errors (EXPR)

See this section in zlang for a description of how the *type of an expression* propagates up an expression tree to its root. The remaining discussion is specific to `EXPR` semantic checks.

Expression Operation Rules (generate `EXPR ERROR`).
unary `PLUS` and binary `PLUS`, `TIMES`, and `BOOLS` (`<`, `<=`, `==`, `>=`, `>`) cannot be used on `string` or `bool` values
`mod` (modulo arithmetic) operation must be used on `int` values (both operands)
`compl` (bitwise complement) unary operation must be used on an `int` value
`not` (logical not) unary operation must be used on a `BEXPR` or `bool` value
treating a function identifier as if it were a value (variable) identifier

Is it ever safe to stop looking for `EXPR` errors in poorly formed expression trees? Evaluation of an expression tree works from the bottom up (recursion is used to get *down to the bottom*).

1. Each node should look for `EXPR` errors under each child branch.
2. When an `EXPR` error occurs in any child, a parent node can no long evaluate the correctness of its operation and so should not emit any `EXPR` errors.
3. If none of a parent node's children report `EXPR` errors, the parent should evaluate the correctness of its operation.
4. Except for the root, all nodes communicate either an error condition or a resultant value type for its operation to its own parent.
5. `domain` nodes in an expression tree simply propagate their child node's value type (domain) or an `EXPR` error condition to their parent. DOMAIN OUTPUT lines should only be generated when there are no `EXPR` errors from the child node.

Keep in mind that all identifiers on the RHS of an assignment (`=`) should be considered "used", whether their use is semantically correct or not, see UNUSED.

Likewise all non-function identifiers on the LHS of an assignment (`=`) should be considered "initialized", whether the RHS is semantically correct or not, see UNINIT.

OUTPUT

`ZOBOS` will report four types of data to three different files.

Syntax, Warning and Error Messages

These will be written to system `stdout` and **must be** prefixed by the token `OUTPUT` as described by the course submission requirements. These messages will have four critical parts emitted in this order:

1. the type of message `:TYPE:`, enclosed with colons[1] where `TYPE` is one of (all capitals) `SYNTAX`, `ERROR`, or `WARN`.
2. the line number of the error in the source (see *Terminal Location* column of the syntax, error, and warning table)
3. the character number of the error in the source (see *Terminal Location* column of the syntax, error, and warning table)
4. the ID for the message type

The line and character number come from the `program.tok` **token stream**. For a `SYNTAX` error it is the location of the offending token *or* the the last good token *iff* the syntax error occurs because there are no more tokens to be parsed. For `WARN` and `ERROR` messages, the location to report is detailed here.

Example Syntax Error Output

Since we abort all tasks on a syntax error, there should only be one `SYNTAX` error ever printed on `stdout`. You **are permitted to print other information**, just don't put it on the required `OUTPUT` line:

`danglingbrace.vrb` (the source `danglingbrace.syn` is available in the ZOBOS archive files)

```
OUTPUT :SYNTAX: 11 1 :SYNTAX:
Parsing error in state 155; input: rbrace; expected=bool|const|emit|
float|id|if|int|rand|string|while
```

> You aren't required to print out anything beyond the data on the `OUTPUT` line.

Expression Tree DOMAIN Messages

These will be written to system `stdout` and **must be** prefixed by the token `OUTPUT` as described by the course submission requirements.

The following are examples of DOMAIN messages from `domain.vrb`:

DOMAIN lines from `domain.vrb` (the source `domain.src` is available in the ZOBOS archive files)

```
OUTPUT :DOMAIN: 1 9 :int:
OUTPUT :DOMAIN: 2 11 :float:
OUTPUT :DOMAIN: 3 10 :bool:
OUTPUT :DOMAIN: 4 12 :string:
```

The line and column number for `DOMAIN` messages are the location of the `domain` terminal in the source and provided to your parser in the token steam file (`.tok`).

Abstract Syntax Tree

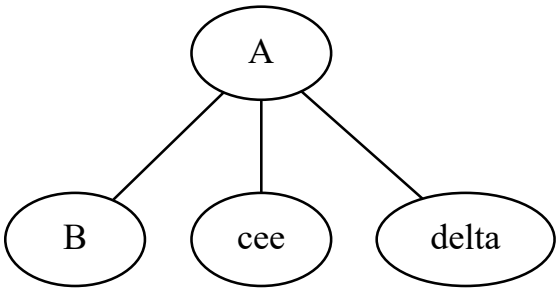
> You have the option of writing a `dot(1)` instruction file **instead of** the format described below. The `grader.sh` script will auto-detect the format you use.

`ZOBOS` will write its abstract syntax tree to the filename specified by the second command line parameter. The format of this file is in two parts. The **first part** is a line by line declaration of a tree node `id` and `name`, separated by white space. If a name is not provided the `id` is used.

An **empty line** separates the the first part from the second part.

The **second part** is a line by line description of tree edges. The first id on the line is the parent, subsequent ids are the children in left to right order. If a parent id is mentioned more than once, children are simply arranged in left to right, file order. The following two AST descriptions in this format yield the same tree graph:

First AST Description	Second AST Description	`dot()` Generated Graph

`A A`
`B`
`C cee`
`D delta`
 	
`A B C D`
	`A A`
	`B`
	`C cee`
	`D delta`
	
	`A B`
	`A C D`
		

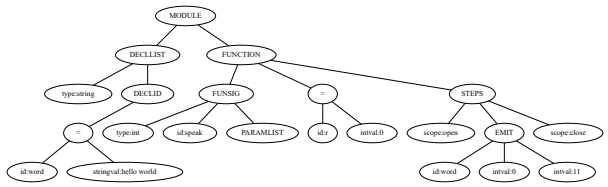
Source (helloworld.src)	AST Description (helloworld.ast)

`string word = "hello world";`
`int speak()`
`returns r = 0`
`{`
`  emit word, 0, 11;`
`}`
	`0 MODULE`
	`0-0 DECLLIST`
	`0-0-0 type:string`
	`0-0-1 DECLID`
	`0-0-1-0 =`
	`0-0-1-0-0 id:word`
	`0-0-1-0-1 stringval:hello world`
	`0-1 FUNCTION`
	`0-1-0 FUNSIG`
	`0-1-0-0 type:int`

```
0-1-0-1 id:speak
0-1-0-2 PARAMLIST
0-1-1 =
0-1-1-0 id:r
0-1-1-1 intval:0
0-1-2 STEPS
0-1-2-0 scope:open
0-1-2-1 EMIT
0-1-2-1-0 id:word
0-1-2-1-1 intval:0
0-1-2-1-2 intval:11
0-1-2-2 scope:close

0-0-1-0 0-0-1-0-0 0-0-1-0-1
0-0-1 0-0-1-0
0-0 0-0-0 0-0-1
0-1-0 0-1-0-0 0-1-0-1 0-1-0-2
0-1-1 0-1-1-0 0-1-1-1
0-1-2-1 0-1-2-1-0 0-1-2-1-1 0-1-2-1-2
0-1-2 0-1-2-0 0-1-2-1 0-1-2-2
0-1 0-1-0 0-1-1 0-1-2
0 0-0 0-1
```

`dot()` Generated Graph



> The example above suggests an easy way to create unique identifiers for all AST nodes: the digit patterns are simply the path to each node, and each path element is the child index (with the root node being the exception, its identifier is simply `0`).

Symbol Tables

During symbol table generation and the search for semantic warnings and errors, if `ZOBOS` encounters the `emit symtable` statement all the symbols for all scopes should be written to the **third command line argument**. The format for this output is as follows:

1. The **global** or deepest scope will be considered scope zero (`0`), each nested scope within it is assigned the next sequential scope value. Functions have some particular scoping requirements.
2. Each symbol should be written on one line, and contain the following **comma delimited fields**: `scope value`, `type`, and `id`.
    a. The scope value is a non-negative integer, `0` is used for the deepest or "global" scope.

b. `type` is the sequence of grammar terminals associated with a variable's `DECLTYPE` or it is the **function signature**:

```
returnType//(param1type(/param2type(/param3type(/...))))
```

eg, the function signature for `speak` in the `helloworld.src` example above is simply `int//`.

c. The `id` is the identifier of the symbol.

1. Yes, it is possible to have scopes without any symbols declared within them.
2. Within the same scope, if an identifier is attempted to be re-used,
    a. If required by the semester write-up a REIDENT error should be generated
    b. Regardless of error generation, the attempted re-declaration **is ignored**.

All of the `.src` files with `emit symtable` instructions in the zobos resource archives have `.sym` files provided as well. Here is an example of program source with multiple scoping levels and the `emit symtable` output.

Source (symtable-2.src)

```
string  m = "helloworld";
const float  pi = 3.1415;
int     i = 0;
const bool    b = 1<3;
int main()
returns r=0
{
    string p = m;
    int    f = i;
    string i = "eyespymeashadow";
    {
        {
            bool c = b;
            emit p, 0, int(b);
            if ( f > 9.81 ) {
                c = bool(f*f-int(c));
                emit symtable;
            }
            emit i, 0, 15;
        }
        emit m, int(pi), 5;
    }
}
```

Symbol Table Dump (symtable-2.sym)

```
0,const bool,b
0,int,i
0,const string,m
0,const int//,main
0,const float,pi
1,int,r
2,int,f
2,const string,i
2,const string,p
4,bool,c
```

Hints and Testing

1. You know you're going to dump symbol table debug info during development, so you might as well format it according to the requirements in the first place.

grader.sh

I am providing to students the same grader script that will be used on gradescope for testing your `ZOBOS`. Here is how to use it:

First, fire up your usual course development environment and download this tarball to your development environment and unroll it in a temporary directory.

```
$ ls zobos-student.*
zobos-student.tar.bz2
$ mkdir ~/tmp
$ cd ~/tmp
$ tar xjf ../zobos-student.tar.bz2
:
$ ls
zobos/
```

Now go to the directory holding your `ZOBOS` and execute the `grader.sh` script from the `zobos-student.tar.bz2` resource.

```
$ cd ~/compilers/zobosCC
$ ls ZOBOS
ZOBOS
$ ~/tmp/zobos/grader.sh
:
:
:
```

This is a *big project* with lots of tests. Some of you might like to tackle the requirements in a more structured "one step at a time" approach, as opposed to the default "test everything always" approach of `grader.sh`'s default invocation. You can use the `GRADERTEST` environmental variable to accomplish this:

```
$ GRADERTEST=COMPILE ~/tmp/zobos/grader.sh
$ GRADERTEST=SYMTABLE ~/tmp/zobos/grader.sh
$ GRADERTEST=AST ~/tmp/zobos/grader.sh
$ GRADERTEST=SEMANTIC ~/tmp/zobos/grader.sh
```

These individual invocations will test only compilation exit status results, symbol table tests, AST generation and finally `DOMAIN`, `WARN` and `ERROR` `OUTPUT`. See the `test_thewhole_shebang` function in `grader.sh` for the gory details. Incidentally, you can also do

```
$ GRADERTEST='AST|SEMANTIC' ~/tmp/zobos/grader.sh
```

to test (you guessed it) AST generation and `OUTPUT`. **Note the quoting of the GRADERTEST value!**

If you are enrolled for graduate credit, set the `GRADERMODE` environmental variable to `ALL`

before running the grader script. The grader script will then test against the "All submissions" and the "Graduate submissions" targets as specified in the the syntax, error and warning table. The default (if `GRADERMODE` is not found in the environment or `GRADERMODE=ZOBOS`) is to test against only the "All submissions" targets.

```
$ GRADERMODE=ALL ~/tmp/zobos/grader.sh
```

You will need to read any messages from the script carefully, and perhaps hit `ENTER` several times throughout its course. This script checks for:

a. missing data files
b. truncated data files
c. inaccessible data files
d. and the difference between your `ZOBOS` results and expected results.

The latter test results are displayed on the terminal screen, but they whip by pretty quickly so you may need to scroll up and make sure you see them all.

When there is a discrepancy in expected results the script either:

- points you to a data file along side your `ZOBOS` where more failure details are available,
- or all the details are displayed on screen.

Before the `grader.sh` terminates, it will show a summary of some specific rubric line results for the programming project.

> gradescope will use this very same script, possibly with some additional testing `.src` to check your submitted work. If your `ZOBOS` flies through without a hitch, you can likely suspect a good grade for the assignment.

Submit Your Work

Partners

You may complete this assignment in a group of two students **that are both rostered for the same course (undergraduate or graduate credit)**.

Checking and Submitting Your Work

Here are some things to double check your submission against:

i. Your archive does not raise errors when "unrolling" with the `${COMPGRADING}/explode-subm` script.

```
$ echo $COMPGRADING
/COMPGRADING
$ ls
```

```
my-project.zip
$ "${COMPGRADING}/explode-subm" -P PROJECT my-project.zip ./here
```

ii. `PROJECT` is different for each project, the first two (for example) are `ALPHABETENCODING` and `LUTHOR`.
iii. Your archive contains only the essential files, **don't provide unneeded files**
iv. Don't provide files that have been provided to you (such as test cases in the `*-student.tar.bz2` files); that would just be silly.
v. Make sure the `grader.sh` script for the assignment runs to completion.

> You aren't required to use `.zip` files, `explode-subm` knows about the usual suspects: `.tar`, `.tar.gz`, `.tar.bz2` ...

When you are happy with your work, submit your project archive file for grading at https://www.gradescope.com.

Rubric

This work is worth 120 points.

Requirements	Points	Notes
Meets compilers course project requirements	10	
Basic I/O and exit status requirements	10	**exit status 1** on unreadable input files **or** un-writable output files
Detect and report `SYNTAX` errors correctly	20	**exit status 2** when `SYNTAX` errors are detected
Output of AST graph information	10	
DOMAIN requirements	20	
`emit symtable` and `symtable.dat` requirements	20	
`ERROR` class semantic issues properly detected and reported	10	**exit status 3** used when `ERROR` issues are detected
`WARN` class semantic issues properly detected and reported	10	**exit status of 0** when only `WARN` issues are detected
AST simplification of control structures `IF`, `IFELSE`, `WHILE`, `DOWHILE`, `STEPS`	10	Manually graded. This **requires** valid AST graph output!

1. the course submission requirements again. (1)

Assignments/ZOBOS (last edited 2026-04-25 09:17:05 by khellman)